

Sprawozdanie: Implementacja publicznego systemu kryptograficznego w oparciu o algorytm RSA

Jakub Biernat 160248

1. Założenia – jak duże liczby pierwsze mogą być wykorzystane w programie?

W moim programie używam liczb pierwszych o dowolnej podanej przez parametr wywołania długości bitów, jednak w celach testowych wykorzystywałem p i q długości 32-64 bitów.

2. Opis metod użytych do wyznaczania e i d

a. wyznaczenie e :

Do generowania losowych liczb e używałem funkcji:

`random.randint(1, phi - 1)`

a do sprawdzania czy jest względnie pierwsza z ϕ użyłem porównania:

`math.gcd(e, phi) == 1`

b. wyznaczenie d :

Do wyznaczenia d użyłem funkcji:

`pow(e, -1, phi)`

3. Opis realizacji zadań (programu i jego składowych) i wartości uzyskane podczas ich realizacji

I. Generowanie liczb pierwszych p i q :

Funkcja `generate_prime(bits)` generuje liczby pierwsze o zadanej długości bitów

II. Generowanie kluczy :

Funkcja `rsa(p, q)` zwraca pary kluczy (e, n) oraz (d, n)

III. Szyfrowanie:

Funkcja `rsa_cipher(text, public_key)` odczytuje znaki z pliku tekstowego `text`, zamienia je na kod ASCII, a następnie szyfruje przy użyciu klucza `public_key`, wypisuje oraz zwraca zaszyfrowany tekst

IV. Deszyfrowanie:

Funkcja `rsa_decipher(encrypted, private_key)` odszyfrowuje tekst `encrypted` przy użyciu klucza `private_key`, a następnie wypisuje go na konsoli

V. Wartości uzyskane podczas testu (p i q długości 50 bitów):

p: 565112760052493

q: 938276162630813

e: 151588409199150037201529391909

d: 374389878282750813905522108173

n: 530231831955760526078859266809

Tekst zaszyfrowany:

[415122001380869152054342647022,
502346260350593790562727131960, 348541900370497665679042076514,
64142958123715157281065466150]

Tekst odszyfrowany:

1234

4. Odpowiedzi na pytania

a. Jakie elementy algorytmu są trudne w realizacji?

Najtrudniejszy moim zdaniem element to generowanie dużych liczb pierwszych, które przy dużej ilości bitów jest kosztowne obliczeniowo.

b. Co stanowi o bezpieczeństwie i jakości tego algorytmu szyfrowania?

O bezpieczeństwie tego algorytmu stanowi ogromna trudność w rozłożeniu n na p i q , natomiast odwrotna operacja jest bardzo szybka.

5. Wnioski

Algorytm RSA jest prostym w implementacji i działającym poprawnie algorytmem generowania kluczy do szyfrowania asymetrycznego, trudność stanowi natomiast jego złożoność obliczeniowa, umożliwiającą wykorzystanie bardzo dużych liczb pierwszych, oraz szyfrowanie długich wiadomości w rozsądnym czasie.